

Vaija - Mapa Tecnico por Arquivos e Pastas

Visao estrutural do projeto existente no workspace.

Resumo: frontend em React + TypeScript + Vite, backend em Node + Express, persistencia em Postgres, scripts locais em PowerShell, deploy com Render/Vercel e automacao n8n para notificacoes de status de pedidos.

```

vaija/
├─ src/                                Frontend React + TypeScript
│   ├── main.tsx                      Entrada da aplicacao
│   ├── App.tsx                       Casca principal da app
│   ├── index.css / App.css           Estilos globais
│   │
│   ├── routes/
│   │   ├── app-routes.tsx            Mapa de rotas publicas e privadas
│   │   └─ protected-route.tsx       Guarda de autenticacao
│   │
│   ├── contexts/
│   │   └─ auth-context.tsx          Estado global de login, usuario e
sessao
│   │
│   ├── pages/                        Paginas de nivel de rota
│   │   ├── landing/landing-page.tsx
│   │   ├── login/login-page.tsx
│   │   ├── dashboard/dashboard-page.tsx
│   │   ├── orders/orders-page.tsx
│   │   ├── pos/pos-page.tsx
│   │   ├── menu/menu-page.tsx
│   │   ├── reports/reports-page.tsx
│   │   └─ settings/settings-page.tsx
│   │
│   ├── components/                  Blocos reutilizaveis de interface
│   │   ├── layout/                  Estrutura do painel
│   │   │   ├── admin-layout.tsx
│   │   │   ├── app-header.tsx
│   │   │   ├── app-sidebar.tsx
│   │   │   └─ page-container.tsx

```

[illegible]

└ hero.png	
└ react.svg	
└ vite.svg	
└ server/	Backend Node + Express
└ └ index.js	Bootstrap do servidor
└ └ app.js	Configuracao do Express e rotas
└ └ routes/	Endpoints HTTP
└ └ └ auth-routes.js	
└ └ └ orders-routes.js	
└ └ └ catalog-routes.js	
└ └ └ users-routes.js	
└ └ services/	Regras de negocio
└ └ └ auth-service.js	
└ └ └ orders-service.js	
└ └ └ catalog-service.js	
└ └ └ order-status-notifier.js	
└ └ repositories/	Acesso e persistencia no Postgres
└ └ └ users-repository.js	
└ └ └ orders-repository.js	
└ └ └ products-repository.js	
└ └ middlewares/	Regras transversais
└ └ └ auth-middleware.js	
└ └ └ permission-middleware.js	
└ └ validators/	Validacao de payloads
└ └ └ order-validator.js	
└ └ └ product-validator.js	
└ └ lib/	Infra compartilhada
└ └ └ db.js	Conexao com Postgres
└ └ └ env.js	Leitura das variaveis de ambiente
└ └ └ cookies.js	Cookies de autenticacao
└ └ └ permissions.js	Permissoes por papel
└ └ data/	Seeds/mocks do backend
└ └ └ mock-users.js	
└ └ └ mock-orders.js	
└ └ └ mock-products.js	
└ public/	Assets publicos servidos pelo Vite
└ └ favicon.svg	
└ └ icons.svg	

```

├─ docker/
│   └─ postgres/init/
│       └─ 001-create-database.sql Bootstrap inicial do banco
│
├─ scripts/                                Operacao local e banco
│   ├── start-local.ps1
│   ├── stop-local.ps1
│   ├── backup-postgres.ps1
│   └─ restore-postgres.ps1
│
├─ n8n/
│   └─ order-status-whatsapp.json Workflow de automacao WhatsApp
│
├─ dist/                                    Build gerado do frontend
├─ docker-compose.yml                      Stack local
├─ docker-compose.prod.yml                Stack de producao
├─ render.yaml                             Deploy no Render
├─ .env / .env.example                    Variaveis de ambiente
├─ vite.config.ts                          Configuracao do Vite
├─ tsconfig*.json                         Configuracao TypeScript
├─ package.json                            Dependencias e scripts
└─ README.md                              Visao geral e instrucoes

```

Conexao entre camadas

1. `src/pages/*` chama `src/lib/*-api.ts`
2. `src/lib/api-client.ts` envia requests para `/api`
3. `server/routes/*` recebe a requisicao
4. `server/services/*` aplica regra de negocio
5. `server/repositories/*` grava ou le no Postgres
6. `order-status-notifier.js` aciona o n8n quando o pedido muda de status

Arquivo gerado automaticamente a partir da estrutura atual do projeto Vaija.